

AI Agents Cheat Sheet

19 tables · 184 concepts · Updated 2026-09-27 · [cheatgrid.com](#)

AI agents are systems in which a large language model directs its own process and tool use in a loop, picking each next step from tool results. Anthropic's engineering guide separates them from **workflows**, where LLMs and tools follow predefined code paths, and suggests an agent only when the number of steps cannot be predicted in advance. The ecosystem is settling around shared pieces: MCP and AGENTS.md moved to the Linux Foundation's Agentic AI Foundation in December 2025, and Microsoft Agent Framework now succeeds both AutoGen and Semantic Kernel. A useful mental model is that an agent is just an LLM using tools on environmental feedback in a loop, so most of the engineering goes into **what enters its context** at each step, **when the loop stops**, and which actions pause for a human.

CONTENTS

Core Agent Concepts	1
Tool Integration	2
Reasoning Techniques	4
Planning Strategies	5
Architecture Patterns	5
Memory Systems	6
Context Management	7
Retrieval-Augmented Generation (RAG) for Agents	8
Multi-Agent Systems	9
Communication Protocols	10
Agent Frameworks	11
State Management	12
Execution Patterns	13
Error Handling	13
Evaluation & Testing	14
Observability & Debugging	15
Security & Safety	16
Cost Optimization	18
Production Patterns	19
References	19

TABLE 1

Core Agent Concepts

These are the building blocks every AI agent shares regardless of framework: the augmented LLM (a model plus retrieval, tools and memory), a control loop that thinks-acts-observes, structured tool invocation, a reasoning engine, perception of inputs, and a calibrated level of autonomy. Grasp these and the framework-specific patterns in later tables become small variations on the same theme.

Concept	Example	Description
Augmented LLM	LLM + retrieval + tools + memory	• An LLM enhanced with retrieval, tools and memory, the foundational building block every agent is built on • the model actively drives these augmentations itself: generating its own search queries, picking tools, and deciding what to retain

Concept	Example	Description
Agent Loop	<pre>while not done and steps < max_steps: thought = think(observation) action = decide(thought) observation = execute(action)</pre>	• The think-act-observe cycle: the agent reasons, acts, and observes environmental feedback each turn • runs until the task completes or a stopping condition, such as a max iteration count, ends it
Tool Use	<pre>tools = [calculator, web_search, db_query] agent.invoke("What's 2^16?", tools)</pre>	• Broad pattern where the agent invokes external functions or APIs beyond text generation, with the runtime (not the LLM) executing the call and feeding the result back • tools are defined with name, description, and JSON schema for parameters
Function Calling	<pre>{"name": "get_weather", "args": {"city": "NYC"}}</pre>	• LLM-side mechanism that outputs structured JSON specifying which function to call and with what arguments • the model only proposes the call; the application or agent runtime is what actually executes it and returns the result
Agentic Workflow	Goal → Plan → Execute → Observe → Adjust → Repeat	• Dynamic, goal-driven process where the agent decomposes the task, takes actions, evaluates outcomes, and adapts its strategy • distinct from a linear prompt chain, which follows fixed predefined steps and is a workflow, not an agent
Perception	<pre>raw_input = {"text": query, "context": session} structured_data = parse(raw_input)</pre>	• Transforms raw inputs (text, API responses, sensor data) into structured representations the reasoning engine can act on • includes parsing, normalization, filtering noise, and entity extraction, more than just receiving text
Reasoning Engine	LLM + prompting strategy (ReAct/CoT) + memory + planning	• Core decision-making component that weighs inputs, selects actions, and generates plans for the agent • typically an LLM operating inside a scaffold of memory, planning, and prompting strategies, since the agent is the orchestrated system, not the bare LLM
Action Space	<pre>["search[entity]", "lookup[string]", "finish[answer]"]</pre>	• Set of available tools and operations the agent is allowed to invoke • defines the boundary of what the agent can accomplish in its environment • tools outside this set cannot be selected by the LLM
Computer-Using Agent (CUA)	<pre>agent.screenshot() agent.click(x=320, y=240) agent.type("search query")</pre>	• Uses vision to read the screen, then executes clicks and keystrokes to control any GUI without a dedicated API • vision-only CUAs (OpenAI, Azure) handle any app, while DOM-aware variants (browser-use, UFO) add accuracy on narrow targets
Autonomy	Agent decides when to stop vs. asks user for approval per action	• Degree to which the agent makes decisions without human intervention, measured as a spectrum rather than a binary • ranges from fully autonomous to selective human-in-the-loop (HITL) gates on high-stakes actions, the dominant production pattern in 2026

TABLE 2

Tool Integration

Function calling tells you *that* an LLM can ask for a tool; this table covers the runtime plumbing that actually makes tools work. It maps the lifecycle from defining the schema, discovering what's available, executing the call, parsing the result, and scheduling multiple calls in parallel or in sequence, including the efficient code-execution pattern for large MCP server ecosystems.

Technique	Example	Description
Tool Schema	<pre>{"name": "get_weather", "description": "...", "parameters": {...}}</pre>	• JSON-Schema description of the tool's name, when to use it, and its parameter shape • the model reads this metadata to decide whether and how to call the tool. The implementation code lives in your runtime, never in the schema
Tool Design (ACI)	<pre>"path": "/repo/src/main.py" # reject relative paths like "src/main.py"</pre>	• Writing a tool's schema and description for the model to read: example usage, edge cases, clear boundaries from similar tools • poka-yoke arguments, for example requiring absolute file paths instead of relative ones

Technique	Example	Description
Hosted Tools vs Function Tools	<pre>tools=[WebSearchTool(), CodeInterpreterTool()] @function_tool def get_weather(city: str): ...</pre>	• Hosted tools (web search, file search, code interpreter, hosted MCP) run on the provider's own servers, no code required • function tools wrap your own functions and run in your process
Tool Discovery	<pre>tools = mcp_client.list_tools_sync()</pre>	• Runtime enumeration of available tools via a <code>tools/list</code> request over JSON-RPC • lets agents pick up new capabilities without redeployment, but every advertised schema costs context tokens
Tool Execution	<pre>name, args = parse(tool_call) result = tools[name](**args)</pre>	• Agent runtime invokes the function the LLM selected, parsing the call name and JSON arguments • the LLM only generates the call; if your code skips dispatch, the model fabricates an observation instead
Tool Result Parsing	<pre>obs = clean(tool_output) messages.append({"role": "tool", "content": obs})</pre>	• Converts raw tool output into a clean observation message the LLM can reason over • typically JSON-stringifies structured data and strips noise like rate-limit headers, request IDs, and other internal metadata
Tool Error Handling	<pre>{"type": "tool_result", "tool_use_id": "toolu_01...", "content": "Error: file not found", "is_error": true}</pre>	• Reports a failed call back to the model as a normal message instead of crashing the loop, so it can retry or pick another tool • Claude's <code>tool_result</code> blocks set <code>is_error: true</code> ; other SDKs expose an equivalent hook
Structured Output	<pre>response_format={"type": "json_schema", "strict": true, "schema": {...}}</pre>	• Constrained decoding masks any token that would violate the supplied JSON Schema, guaranteeing schema-conformant output • stronger than the older JSON mode (which only guarantees valid JSON syntax) • supported by OpenAI, Anthropic, Google, and AWS Bedrock; guarantees shape, not semantic correctness
Parallel Tool Use	<pre>calls = [get_weather("NYC"), get_weather("LA")] results = await asyncio.gather(*calls)</pre>	• The model may emit multiple tool calls in one turn, and the runtime dispatches them concurrently • works only when calls share no data dependency: hidden dependencies (e.g. fetch-then-update on shared state) create race conditions
Tool Chaining	<pre>user_id = get_user(name) orders = get_orders(user_id)</pre>	• Sequential composition where the output of tool A feeds the input of tool B, across successive turns of the tool-use loop • the data dependency forces serial execution and is the structural opposite of parallel tool use
Tool Search (Deferred Loading)	<pre>@tool(defer_loading=True) def get_customer_profile(custo mer_id: str): ... agent = Agent(tools=[*crm_tools, ToolSearchTool()])</pre>	• Defers loading of a large tool surface until the model actually needs it, instead of listing every schema on every turn • pairs <code>defer_loading</code> tools or namespaces with one <code>ToolSearchTool()</code> ; cuts tool-schema tokens on toolsets too large to load upfront
Code Execution via MCP	<pre>import * as gdrive from './servers/google-drive' const doc = await gdrive.getDo cument({documentId})</pre>	• Presents MCP servers as code APIs on a filesystem instead of direct tool calls, loading only the tool definitions the agent needs • keeps intermediate results out of the model context, cutting token usage by up to 98.7%

TABLE 3

Reasoning Techniques

Reasoning techniques shape how an agent thinks through a problem before it acts, from solving with no examples (zero-shot) to spending extra inference-time compute on hidden reasoning (extended thinking) to looping with the environment (agentic reasoning). The right choice depends on the task's complexity, the cost budget, and whether the agent needs to ground itself in external observations.

Technique	Example	Description
Chain-of-Thought	Let's think step-by-step: 1. Parse question 2. Identify data needed 3. Calculate result	- Prompts agent to show intermediate reasoning steps - improves accuracy on complex tasks by making thought process explicit before answering.
Zero-Shot	"Translate this to French: Hello"	- Agent solves the task with no examples in the prompt, relying solely on instruction and pre-training - no weight updates, distinct from fine-tuning - fastest and cheapest, but less reliable for complex or ambiguous tasks.
Few-Shot	Examples: Q: 2+2 A: 4 Q: 3+5 A: 8 Now: 7+9 = ?	- A handful of input-output example pairs are placed in the prompt before the actual query - the model conditions on these demonstrations in-context, with no gradient updates (Brown et al., GPT-3, 2020) - teaches task format and output style purely through demonstration.
Reasoning Models	<code>response = client.responses.create(model="gpt-6-astra", reasoning={"effort": "low"}, input=[...])</code>	- Model spends hidden reasoning tokens working through the problem before it answers - a reasoning effort setting (low/medium/high) trades latency and cost for depth of analysis.
Thinking Budget	<code>thinking={"type": "enabled", "budget_tokens": 10000}</code>	Numeric cap on how many tokens a model may spend on internal reasoning before its final answer - a target, not a hard limit: minimum 1,024 tokens, and must stay below <code>max_tokens</code>.
Self-Consistency	Run the same CoT prompt 5 times with <code>temperature > 0</code> , return the majority answer	- Samples multiple independent chain-of-thought paths for the same question, then returns the answer that appears most often (Wang et al., 2022) - replaces greedy decoding with sample-and-vote aggregation - boosts hard-reasoning accuracy (e.g. +17.9% on GSM8K) at roughly N× the token cost.
Tree-of-Thought	Explore multiple reasoning paths in tree structure, backtrack if needed	- Agent explores multiple solution paths simultaneously, evaluating each branch before committing - useful when single linear reasoning path may fail.
Graph-of-Thought	Nodes = LLM thoughts, edges = dependencies; aggregate, refine, loop	Generalizes Tree-of-Thought to an arbitrary graph of thoughts (Besta et al., 2023), letting branches merge, cycle and refine each other - trades higher orchestration cost for quality gains on elaborate problems.
Agentic Reasoning	Thought → Action → Observation → Thought → ... (ReAct loop)	Closed-loop reasoning: the agent thinks, takes an action, observes the outcome, and adjusts the next thought - canonical instantiation is ReAct (Yao et al., 2022), which grounds reasoning in real observations and cuts hallucination versus pure chain-of-thought.
Interleaved Thinking	Tool call → result → [thinking: assess result] → next tool call	- Lets the model reason between tool calls within a single turn, weighing each result before deciding what to do next - automatic on adaptive-thinking models; needs a beta header in manual mode on older ones.

TABLE 4
Planning Strategies

How an agent turns a goal into steps and revises them as execution unfolds. Strategies differ in when the plan is built, how granular it is, and what the agent does when reality does not cooperate, from one-shot decomposition and hierarchical breakdowns, through runtime replanning and persistent to-do tracking, to the upfront Planner-Worker-Solver split of ReWOO and the parallel plan execution of LLMCompiler.

Strategy	Example	Description
Task Decomposition	"Write report" → ["research", "outline", "draft", "edit"]	• Breaking complex goal into subtasks • agent identifies logical steps required to achieve objective, each simpler than original.
To-Do List Tracking	pending → in_progress → completed (per todo item)	• Agent maintains a persistent task list it writes and updates as it works through a multi-step plan • each item moves through a lifecycle, giving visibility into long-running work.
Dynamic Replanning	Adjust plan when action fails or new info appears	• Agent updates strategy based on execution results • abandons unsuccessful paths and generates new plans in response to changing conditions.
Hierarchical Task Network (HTN)	Goal decomposed via methods → subtasks → ['boil_water', 'add_tea_leaves', 'pour_tea']	• Multi-level decomposition: abstract tasks are broken into subtasks by domain-specific methods, recursively, until only primitive, directly executable actions remain • refines a top-level goal into an ordered sequence of tactical steps.
ReWOO	Planner builds a plan → Worker calls tools for evidence → Solver produces the answer, no intermediate observations needed	• Reasoning WithOut Observation, separates planning from tool execution • a Planner creates the complete plan before any tool is called, so Workers run without interleaving reasoning steps • more token-efficient than ReAct for predictable workflows.
LLMCompiler	Function Calling Planner builds the plan → Task Fetching Unit dispatches ready calls → Executor runs them in parallel	• Parallel plan execution: an upfront plan lets independent tool calls run concurrently instead of one at a time • cuts latency and cost versus sequential, one-call-at-a-time function calling.

TABLE 5
Architecture Patterns

These patterns describe how a single agent system organizes control flow, from fixed workflows where predefined code paths run LLM calls in sequence, to autonomous agents where the model itself decides what happens next. ReAct decides one step at a time, Plan-and-Execute commits to a full plan upfront, and Reflection critiques its own output before continuing.

Pattern	Example	Description
ReAct	Thought: Need price data Action: query_db("SELECT price") Observation: \$42 Thought: Now calculate...	• Reasoning and Acting interleaved • agent generates Thought explaining reasoning, executes Action, receives Observation of result, then generates next Thought • think-act-observe pattern improves reliability and debuggability.
Prompt Chaining	out1 = llm(step1_prompt) if not gate_check(out1): stop out2 = llm(step2_prompt(out1))	• Decomposes a task into a fixed sequence of LLM calls, each processing the previous call's output • a programmatic gate checks intermediate steps to keep the process on track • trades latency for accuracy on tasks that decompose cleanly.
Router	if query_type == "sql": route_to(sql_agent) else: route_to(general_agent)	• Conditional dispatcher that classifies an input and directs it to a specialized followup task or agent • enables separation of concerns and more specialized prompts than one generalist call.

Pattern	Example	Description
Parallelization	<pre>sections = [llm(part) for part in subtasks] votes = [llm(same_task) for _ in range(3)]</pre>	- Runs multiple LLM calls simultaneously, then aggregates the outputs programmatically Sectioning splits a task into independent subtasks; Voting runs the same task repeatedly for diverse outputs and higher confidence.
Reflection	<pre>output = generate() critique = evaluate(output, criteria) if not good: revise(critique)</pre>	- Agent self-evaluates outputs against quality criteria, then iterates to improve - generates, critiques, and refines until meeting standards or max attempts.
Evaluator-Optimizer	<pre>output = generator.propose(task) feedback = evaluator.critique(output, criteria) if not done: generator.revise(feedback)</pre>	Generator produces output, evaluator critiques against criteria, loop repeats until quality threshold met - enables autonomous quality assurance via two cooperating agents instead of one.
Orchestrator-Worker	<pre>supervisor.assign(task, worker_agents) results = await all_workers() supervisor.synthesize(results)</pre>	Hierarchical delegation where supervisor breaks work into parallel subtasks, assigns to worker agents, aggregates results - scales complex workflows efficiently.
Plan-and-Execute	<pre>plan = planner.create(goal) for step in plan: execute(step)</pre>	Separates planning from execution - planner creates full task breakdown upfront, then executor runs each step - clearer than pure ReAct for multi-step workflows.

TABLE 6

Memory Systems

Memory turns stateless LLMs into agents that learn across sessions. This table maps the canonical short-term vs long-term split, the three cognitive long-term types (episodic, semantic, procedural), the techniques that get information in and out of a context window across runs, and production runtimes like Mem0, Zep/Graphiti and Letta (MemGPT).

Type	Example	Description
Short-Term Memory	Current conversation context in prompt	Thread-scoped context for the active session - holds recent messages and intermediate outputs inside the model's context window; cleared when the thread ends.
Long-Term Memory	Vector DB storing past interactions	Persistent storage across threads and sessions - agent retrieves relevant historical context to inform current decisions - critical for maintaining user preferences and learning.
Working Memory	Variables tracking current task state	Active scratchpad for the current task - holds intermediate results, loop counters, and task-specific variables that the agent manipulates while reasoning, distinct from raw conversation history.
Episodic Memory	"On 2026-01-15, user preferred JSON output"	- Stores specific past events tied to a time and place - lets the agent recall "what happened when" instead of only "what is true in general."
Semantic Memory	"User always wants reports in PDF format"	- Stores durable facts, preferences, and learned knowledge that are true in general - extracted and consolidated from episodic traces into atomic, deduplicated entries.
Procedural Memory	Learned workflows like "how to file a bug report"	- Encodes how to perform tasks: communication styles, formatting rules, action sequences - captured from feedback and reused as the agent's default behavior.
Shared Memory	<pre>redis.set("team_context", state) other_agent.get("team_context")</pre>	Cross-agent state enabling coordination - multiple agents read/write a common store (Redis, shared DB) so they stay aligned in multi-agent systems.

Type	Example	Description
Structured Note-Taking	agent writes progress to NOTES.md rereads the file after a context reset	• Agentic memory: the agent regularly writes notes to a file outside the context window and pulls them back in later • keeps critical context and dependencies from being lost across dozens of tool calls.
Memory Namespace	namespace = (user_id, "chitchat") store.put(namespace, "a-memory", {...}) store.search(namespace, query="language preferences")	• Organizes long-term memories under a namespace (like a folder) and a key (like a file name) • supports cross-namespace search through content filters.
Claude Memory Tool	tools=[{"type": "memory_20250818", "name": "memory"}]	• Client-side tool that gives Claude a <code>/memories</code> file directory it can read, write and update • automatically checks that directory before starting a task, so state persists across sessions without re-explaining context.
Mem0	m.add(messages, user_id="u1") results = m.search("user prefs", filters={"user_id": "u1"})	• Dedicated memory layer that dynamically extracts, consolidates, and retrieves salient facts from conversations across sessions • achieves 26% relative improvement over OpenAI's memory and 91% lower p95 latency vs full-context.
Zep/Graphiti	await graphiti.add_episode(name="ep1", episode_body=text) results = await graphiti.search("user prefs", center_node_uuid=uuid)	• Temporal knowledge graph memory storing facts as edges with bi-temporal validity windows • outperforms vector-only approaches on cross-session temporal reasoning (94.8% vs 93.4% on DMR; up to 18.5% gain on LongMemEval).
Letta (MemGPT)	client.agents.messages.create(agent_id, input=[...]) client.agents.blocks.update("human", agent_id=agent_id, value="Prefers JSON output")	• OS-inspired virtual context management with tiered memory (in-context blocks, recall storage, archival storage) • the agent self-edits its own memory blocks through built-in memory tools.

TABLE 7

Context Management

Every agent call runs on a finite token budget: the system prompt, tool schemas, retrieved documents, message history, and the model's own output all share one window. This table covers deciding what enters that window at each step, what gets compacted or cleared as a run grows, and how skills, sub-agents, and instruction files keep only the relevant slice of information in context.

Technique	Example	Description
Context Engineering	Prompt engineering: word the system prompt. Context engineering: also curate tools, history, retrieved docs, MCP each turn	• Curating the whole set of tokens fed to the model, not just the prompt wording • Anthropic frames it as the natural progression of prompt engineering for agents that run over many turns.
Context Window	Claude Opus 4.6 / Sonnet 4.6: 1M tokens (GA); Gemini 2.5 Pro: 1M tokens	• All tokens the model can reference in one call, including the response it generates • covers system prompt, history, retrieved docs, current query, and output • 1M-token windows are now generally available from Anthropic (Opus 4.6 / Sonnet 4.6) and Google.
Context Rot	Needle-in-a-haystack recall accuracy falls as filler tokens fill the window, well before it is full	• Recall and accuracy degrade as the window fills, even below the hard token limit • every added token draws on a finite "attention budget", so context is a resource with diminishing returns.
Context Overflow Handling	Stop with <code>model_context_window_exceeded</code> , then summarize and continue	• Strategies for the hard token limit • modern APIs return an explicit error or stop reason at the boundary rather than silently dropping tokens • common responses are truncation, summarization (compaction), or splitting the work across calls.

Technique	Example	Description
Dynamic Context Selection	Claude Code uses <code>glob</code> and <code>grep</code> to retrieve files just-in-time instead of loading a full index	• Agent decides what to load for the current step instead of stuffing everything up front • keeps the active window small and task-focused, avoiding context rot from irrelevant tokens.
Compaction	<pre>betas=["compact-2026-09-04"] compaction={"type": "summarize"} history = [{"role": "assistant", "content": summary.content}]</pre>	• Summarizes older turns and reinitializes the window with that summary, so no hand-written summarization code is needed • the first lever for keeping a long agent run coherent inside its budget.
Tool Result Clearing	<pre>context_management={"edits": [{"type": "clear_tool_uses_20250919", "trigger": {"type": "input_tokens", "value": 30000}, "keep": {"type": "tool_uses", "value": 3}}]}</pre>	• Clears old tool results once the model has already used them, replacing each with a placeholder • the lightest-touch form of compaction: nothing is rewritten, just dropped.
Summarization Middleware	<code>middleware=[SummarizationMiddleware(model="gpt-4o-mini")]</code>	Prebuilt LangChain middleware that condenses conversation history once it grows too long, alongside PII-redaction and human-in-the-loop middleware in the same hook chain.
Sub-Agent Context Isolation	Lead agent delegates a research task; the sub-agent explores with tens of thousands of tokens, returns a 1,000-2,000 token summary	• Each sub-agent gets its own clean context window for a focused task, then returns only a condensed summary to the lead agent • keeps deep exploration from polluting the coordinator's window.
Agent Skills (Progressive Disclosure)	<pre>--- name: pdf-processing description: Extract text and tables from PDF files, fill forms... ---</pre>	• Skills load in layers: name and description sit in the system prompt at startup, the full <code>SKILL.md</code> body loads only when triggered, bundled files load only when referenced • keeps the amount of packaged knowledge effectively unbounded.
AGENTS.md	<pre># AGENTS.md ## Setup commands - Install deps: npm install - Run tests: npm test</pre>	A single, framework-agnostic instructions file (build steps, tests, conventions) that many coding agents read at the start of a run, kept separate from the human-facing README.
Prompt Compression	LLMLingua drops low-information tokens to shrink a prompt up to ~20x	• Cuts tokens while preserving meaning for the model • LLMLingua-family methods use a small language model to score and drop low-information tokens at the token level • distinct from summarization, which paraphrases rather than removes tokens.

TABLE 8

Retrieval-Augmented Generation (RAG) for Agents

RAG techniques connect an agent to external knowledge so its answers can be grounded in fresh, domain-specific facts instead of memorized weights. The patterns below stack: query transformation reshapes the input, vector or graph retrieval pulls candidates, reranking sharpens them, and an agentic controller decides when any of this is even worth running, or whether to search files directly instead.

Technique	Example	Description
Vector Search	<code>results = vector_store.similarity_search(query, k=5)</code>	• Semantic retrieval of relevant documents using embedding similarity (e.g. cosine, dot product) • agent queries a vector store to augment reasoning with external facts.
Agentic RAG	Agent decides when to retrieve, what to query, how to use results	• Agent controls retrieval rather than always fetching upfront • reasons step-by-step about necessity, formulates queries, may iterate or skip retrieval entirely for simple questions.
Agentic Search	<code>grep -r "handleAuth" src/glob("**/*.test.ts")</code>	• Agent explores files just-in-time with primitives like <code>grep</code> and <code>glob</code> instead of querying pre-computed embeddings • avoids stale indexes, works well when content changes often.

Technique	Example	Description
Query Transformation	Original query → LLM rewrites into a step-back question or several sub-queries → retrieve against those	- Pre-retrieval step that rewrites or expands the query to close the query-document vocabulary gap - includes rewrite-retrieve-read, step-back prompting, and multi-query generation.
Reranking	<pre>compressor = CrossEncoderReranker(model=cross_encoder) retriever = ContextualCompressionRetriever(base_compressor=compressor, base_retriever=base_retriever)</pre>	- Post-retrieval step that re-scores candidates using a cross-encoder that sees query and document jointly - lifts precision after high-recall vector retrieval.
Graph RAG	Query a knowledge graph for entity relationships and community summaries	Retrieves structured knowledge from a graph of entity nodes and relationship edges - supports multi-hop reasoning that flat vector similarity cannot, with substantial gains in answer comprehensiveness on global questions (Microsoft Research, 2024).

TABLE 9

Multi-Agent Systems

Multi-agent systems split work across several agents through a handful of recurring topologies: when control is centralized, when it isn't, and how agents share state. The split between hierarchical (a supervisor delegates downward) and swarm/collaborative (peers interact as equals) is the most-confused distinction, and how state is shared (direct messages vs a shared blackboard) is the second.

Pattern	Example	Description
Hierarchical	supervisor → specialist_a, specialist_b → workers	- A central supervisor delegates tasks to specialized agents, which may themselves supervise sub-teams - communication and routing flow through the supervisor, results aggregate upward - mirrors an org chart and makes ownership and debugging easier than a flat mesh.
Agent-as-Tool	<pre>orchestrator = Agent(tools=[spanish_agent.as_tool(tool_name="translate_to_spanish"), french_agent.as_tool(tool_name="translate_to_french")])</pre>	- A sub-agent is wrapped as a callable tool via <code>agent.as_tool()</code>, invoked like any function call - unlike a handoff, the caller keeps control and gets the result back to use in its next step.
Agent Handoff	<pre>handoff(source=triage_agent, target=billing_agent, condition="billing question")</pre>	- One agent transfers control to another specialized agent mid-conversation - enables seamless delegation where the receiving agent continues with full context.
Sequential	Agent A → Agent B → Agent C (pipeline)	- Agents are chained in a predefined linear order, each processing the previous agent's output - suited to step-by-step workflows where each stage depends on the one before it.
Parallel (Concurrent)	Three agents analyze the same stock concurrently, then a final step merges their answers	- Multiple agents work on independent subtasks at the same time, with no agent reading another's in-progress output - a coordinator aggregates results when they finish, cutting overall latency for decomposable work.
Collaborative	Agents share a scratchpad of messages and discuss until one says FINAL ANSWER	Peer agents interact as equals to solve a problem jointly, with no fixed authority - they exchange messages, debate, and negotiate solutions until they reach consensus.
Group Chat (Manager-Orchestrated)	A chat manager lets a writer, an editor and a fact-checker debate one draft in a shared thread until the editor approves it	- A chat manager arbitrates turn-taking among agents that post into one shared conversation thread - suited to brainstorming, debate and maker-checker review; keep the agent count small and cap iterations to avoid runaway discussion.

Pattern	Example	Description
Loop (Iterative)	<code>LoopAgent(sub_agents=[critic_agent, refiner_agent], max_iterations=5)</code>	- Sub-agents run repeatedly in a fixed order until a max iteration count is hit or a sub-agent signals termination - suited to iterative refinement, such as a draft-critique-revise cycle.
Blackboard	<code>board.write("draft", text)</code> then a reviewer agent triggers when it sees a new draft and writes back <code>board.write("review", feedback)</code>	- Agents read and write to a shared workspace rather than messaging each other directly - each agent watches the board and activates when relevant data appears, enabling loose coupling and emergent coordination without a central orchestrator.
Swarm	<code>create_swarm([alice, bob], default_active_agent="Alice")</code> (each agent hands off control directly to any other)	- A decentralized mesh where agents hand off control to each other based on specialization, with no fixed hierarchy - the system remembers which agent was last active, so the next turn resumes there.

TABLE 10

Communication Protocols

Agents rarely work alone: they reach out to tools and data, hand off work to other agents, and stream results back to users. This table covers the three protocols that now define those boundaries (**MCP**, **A2A**, **AG-UI**) along with the classic messaging patterns (**Pub-Sub**, **Request-Response**, **Message Queue**) that still underpin agent transports.

Protocol	Example	Description
Model Context Protocol (MCP)	<pre>from mcp.server import MCPServer mcp = MCPServer("Demo") @mcp.tool() def add(a: int, b: int) -> int: return a + b</pre>	Open standard originated by Anthropic (Nov 2024), now under the Linux Foundation's Agentic AI Foundation host-client-server architecture over JSON-RPC 2.0; the 2026-07-28 spec made the protocol stateless, dropping session-pinned Streamable HTTP - the agent-to-tools/data layer, distinct from agent-to-agent (A2A)
Agent-to-Agent (A2A)	<pre>message = new_text_message(text_query, role=Role.ROLE_USER) request = SendMessageRequest(message=message) async for chunk in client.send_message(request): print(chunk)</pre>	Inter-agent communication protocol originated by Google, now donated to the Linux Foundation - agents publish agent cards at <code>/.well-known/agent-card.json</code> and exchange task-lifecycle messages to delegate work IBM's ACP was folded into A2A under the Linux Foundation in August 2025
AG-UI	<pre>agent.emit(TextMessageStart(id)) agent.emit(ToolCallStart(id, name))</pre>	Open, event-based protocol for the agent ↔ user-interface boundary, born from CopilotKit's work with LangChain and CrewAI - streams text, tool calls, state, and human-in-the-loop events over SSE, WebSocket, or a binary transport - native support in Bedrock AgentCore and Agent Framework
Pub-Sub	<pre>agent.subscribe("topic/events") publish("topic/events", data)</pre>	One-to-many broadcast pattern: every subscriber to a topic receives its own copy of each message - publishers don't know how many subscribers exist, which decouples senders from receivers - natural fit for event-driven fan-out across many agents
Request-Response	<pre>response = await agent.call(request)</pre>	Synchronous query-reply pattern: the caller blocks until the callee returns a response - simplest model with strong consistency and easy debugging, but produces tight runtime coupling and risks cascading failures - the baseline pattern HTTP / REST inherits
Message Queue	<pre>queue.push(task) worker = queue.pop() worker.execute(task)</pre>	Point-to-point asynchronous delivery: each message is consumed by exactly one worker, FIFO within the queue - decouples producers from consumers and buffers spikes, with retry and dead-letter handling - the workhorse pattern for background work distribution (e.g. SQS, RabbitMQ)

TABLE 11

Agent Frameworks

The agent-framework landscape in 2026 is crowded, and each tool below picks a different bet: graph-based stateful runtimes, role-based crews, conversational multi-agent systems, type-safe structured output, model-driven SDKs, or lightweight code-first harnesses. Use this table to match a project's needs (durability, model-agnosticism, multi-agent style) to the framework whose design philosophy fits best.

Framework	Example	Description
LangGraph	<code>StateGraph</code> with nodes, edges, and checkpoints	• Graph-based orchestration for stateful, cyclical workflows • models agents as state machines with conditional routing and durable persistence via checkpoints • production-grade.
LangChain	<code>create_agent(model, tools=[...])</code> with chains, tools, memory	• Flexible toolkit for building LLM applications • <code>create_agent</code> is LangChain 1.0's standard entry point, customised through middleware (<code>PIIMiddleware</code> , <code>SummarizationMiddleware</code> , <code>HumanInTheLoopMiddleware</code>) • code-first with extensive integrations.
CrewAI	<code>Crew</code> of agents with <code>role</code> , <code>goal</code> , <code>backstory</code>	• Role-based collaboration where agents simulate team dynamics • supports <code>Process.sequential</code> and <code>Process.hierarchical</code> (requires a <code>manager_llm</code> or <code>manager_agent</code>) • fast prototyping for multi-agent workflows.
OpenAI Agents SDK	<code>Agent(name="Assistant", tools=[...])</code> <code>Runner.run_sync(agent, query)</code>	• Official OpenAI framework, the production-ready replacement for the experimental Swarm library • core primitives: Agents, Handoffs, Guardrails, and Agent-as-Tool patterns (plus Sessions, Tracing, and Sandbox Agents for isolated file and shell workspaces) • Python-first with built-in tracing and MCP support.
Claude Agent SDK	<code>async for msg in query(prompt="Fix the bug", options=ClaudeAgentOptions())</code>	• Anthropic's official SDK exposing the same agent harness that powers Claude Code as a library • built-in tools for file reading, command execution, and web search • available in Python and TypeScript.
Microsoft Agent Framework	<code>agent = ChatAgent(chat_client=OpenAIChatClient(model_id="gpt-4"), name="assistant", tools=[get_weather])</code>	• Direct successor to AutoGen and Semantic Kernel, built by the same Microsoft teams • combines AutoGen's simple multi-agent abstractions with Semantic Kernel's enterprise features (session state, type safety, telemetry) • adds graph-based workflows for explicit multi-agent orchestration, for .NET and Python.
Google ADK	<code>SequentialAgent</code> , <code>ParallelAgent</code> , <code>LoopAgent</code> , <code>LlmAgent</code>	• Google's modular framework with workflow agents (Sequential, Parallel, Loop) for deterministic flow and <code>LlmAgent</code> for LLM-driven dynamic routing • ADK 2.0 supersedes these template workflows with graph-based and dynamic workflows for more complex orchestration • model-agnostic, multi-language (Python, Go, Java, TypeScript, Kotlin).
PydanticAI	<code>agent = Agent('anthropic:claude-fable-5', capabilities=[Coder(), WebSearch()])</code>	• Type-safe Python framework by the Pydantic team; the same typed agent loop runs as a CLI or background job • capabilities bundle tools, instructions, and hooks into reusable units, from MCP to a full coding agent • structured output validated end-to-end.
Strands Agents	<code>from strands import Agent</code> <code>agent = Agent(tools=[calculator])</code> <code>agent("What is the square root of 1764")</code>	• AWS open-source SDK with a model-driven approach • model-agnostic, with first-class support for Amazon Bedrock, Anthropic, OpenAI, and Gemini • native MCP support, hooks on every agent-loop step, and a Strands Harness for a batteries-included agent.
smolagents	<code>agent = CodeAgent(tools=[tool], model=model)</code> <code>agent.run("What is the weather?")</code>	• Hugging Face's lightweight agent library built around code-first actions • <code>CodeAgent</code> writes and executes Python snippets as actions (must be sandboxed); <code>ToolCallingAgent</code> is the JSON-tool-call alternative • supports local and remote models.

Framework	Example	Description
Agno	<pre>agent = Agent(model=OpenAIChat(id="gpt-4o")) agent.print_response("Summarize this")</pre>	• SDK for building agent platforms, serving agents through a REST API, an MCP server, or chat apps like Slack • supports memory, knowledge, reasoning, and teams • runs on AgentOS as a durable API and MCP server, monitored through its Control Plane.
goose	<pre>goose session goose run --recipe recipe.yaml</pre>	• Open-source, local-first agent by Block, donated to the Agentic AI Foundation (AAIF) • desktop app, CLI, or API; connects to 70+ extensions over MCP, workflows saved as YAML recipes • works with 15+ model providers, including Claude or ChatGPT subscriptions via ACP.
Semantic Kernel (legacy)	<pre>kernel.add_plugin(MyPlugin(), plugin_name="X") plus planners</pre>	• Microsoft framework for enterprise scenarios, with Azure integration and C# / Python / Java support • plugins are reusable skills the kernel and planners invoke • superseded by Microsoft Agent Framework, built by the same team.
AutoGen (legacy)	<pre>from autogen_agentchat.agents import AssistantAgent agent = AssistantAgent("assistant", OpenAIChatCompletionClient(model="gpt-4o"))</pre>	• Event-driven multi-agent framework built around message passing and agent-to-agent dialogue • now in maintenance mode: no new features or enhancements, community managed going forward • superseded by Microsoft Agent Framework, the recommended starting point for new projects.

TABLE 12

State Management

State management is what lets an AI agent survive crashes, hand off conversations between sessions, and explore alternative trajectories without losing the original run. LangGraph's checkpointer-based persistence and Temporal's event-sourced durable execution are the two dominant approaches in production, with sessions, interrupts, typed schemas and rollback as the supporting mechanics that make either one safe at scale.

Concept	Example	Description
Checkpointing	<pre>graph.compile(checkpointer=PostgresSaver(...)) graph.get_state(config)</pre>	• Saving agent state as a snapshot at every super-step boundary • enables pause/resume, time-travel debugging, and recovery from node failures • critical for long-running agents.
State Persistence	<pre>PostgresSaver.from_conn_string(DB_URI) SqliteSaver, RedisSaver</pre>	• Durable storage of agent state across process restarts • production agents swap <code>InMemorySaver</code> for Postgres/SQLite/Redis backends so threads survive crashes and redeploys.
Thread Management	<pre>config = {"configurable": {"thread_id": "user-123"}} graph.invoke(input, config)</pre>	• Isolating parallel agent sessions • <code>thread_id</code> is the primary key under which checkpoints are stored, giving each user or session independent state with no crosstalk.
Sessions	<pre>session = SQLiteSession("conversation_123") result = await Runner.run(agent, "hi", session=session)</pre>	• Built-in conversation-history persistence scoped to a session ID in an agent SDK • the runner fetches prior turns before each run and stores new ones after, so callers skip manual <code>.to_input_list()</code> bookkeeping.
State Schema	<pre>class AgentState(TypedDict): messages: Annotated[List, add_messages] step_count: int</pre>	• Typed definition of agent state structure (TypedDict, dataclass, or Pydantic BaseModel) • reducers like <code>add_messages</code> tell LangGraph how to merge updates: without them, the last write wins.
Interrupt & Resume	<pre>value = interrupt({"question": "Approve?"}) graph.invoke(Command(resume="yes"), config)</pre>	• Pausing graph execution mid-run to wait for external input, with state saved by the checkpointer at the exact pause point • resuming replays the node from its start, so any code before the interrupt call must be idempotent.

Concept	Example	Description
State Rollback (Fork)	<pre>fork_cfg = graph.update_state(old_cfg, {"x": "new"}) graph.invoke(None, fork_cfg)</pre>	• Forking from an earlier checkpoint with modified state to explore an alternative path • <code>update_state</code> is non-destructive: the original history is preserved, enabling safe "undo" and experimentation.
Durable Execution	<pre>@workflow.defn class AgentWorkflow: async def run(self): ...</pre>	• Agent workflows that survive crashes and restarts via platforms like Temporal • recovery works by replaying the recorded Event History against deterministic workflow code, not by loading a saved state blob.

TABLE 13

Execution Patterns

How an agent is invoked shapes its throughput, perceived latency, and how it plugs into a caller. Synchronous, asynchronous, streaming, background, and event-driven invocation solve different problems and are easy to confuse, especially async vs parallel and streaming vs background polling.

Pattern	Example	Description
Sync Execution	<pre>result = Runner.run_sync(agent, "Plan a trip to Rome") print(result.final_output)</pre>	• Blocking call that waits for agent completion before returning • simpler to reason about but locks the caller until the full response is ready.
Asynchronous Execution	<pre>task = asyncio.create_task(agent.run(query)) result = await task</pre>	• Non-blocking invocation that yields control while the agent works • concurrency (single-threaded event loop), not parallel execution; lets the caller interleave other I/O.
Streaming	<pre>async for token in agent.stream(): print(token, end="")</pre>	• Agent emits partial outputs (token deltas via SSE) as they are generated • cuts Time-to-First-Token and improves perceived latency without changing total generation time.
Background Execution	<pre>resp = client.responses.create(model=m, input=q, background=True) while resp.status in {"queued", "in_progress"}: resp = client.responses.retrieve(resp.id)</pre>	• Fire-and-poll invocation for long-running calls that would otherwise exceed a connection timeout • the caller polls the run object for status instead of holding an open request.
Event-Driven	<pre>agent.subscribe({ onSubagentStartedEvent: (e) => log(e.subagentName), onSubagentErrorEvent: (e) => alert(e) })</pre>	• Callback-based execution where typed events (run lifecycle, tool calls, state deltas, errors) fire registered handlers • canonical pattern in the AG-UI protocol for agent-UI integration.

TABLE 14

Error Handling

Production agents fail constantly: providers throttle, networks blip, models time out, and downstream tools return garbage. These patterns are the standard distributed-systems toolkit applied to LLM agents: retry transient errors with backoff and jitter, return a failed tool call to the model as data it can act on, fall back to alternatives, trip a circuit breaker before a retry storm crashes a recovering provider, degrade gracefully when full output is impossible, treat a run that exhausts its turn budget as a failure, and propagate unrecoverable errors up to a supervisor.

Technique	Example	Description
Retry with Backoff	<pre>for attempt in range(3): try: call_api() except: sleep(2**attempt)</pre>	• Automatic retry with exponentially increasing delays • handles transient failures like rate limits or network glitches.
Return Error to Model	<pre>run_config=RunConfig(tool_not_found_behavior="return_error_to_model")</pre>	• Returns a failed tool call to the model as a result instead of raising an exception • the model reads the error and can pick another tool or retry.

Technique	Example	Description
Fallback Strategies	<pre>model_fallback_list = ["claude-sonnet-5", "gpt-5.6-luna"] for model in model_fallback_list: try: response = completion(model=model, messages=messages) except Exception: continue</pre>	• Alternative approaches when primary fails • agent switches to backup model, tool, or method if first choice unavailable.
Circuit Breaker	After N failures, stop trying for cooldown period	• Prevents cascading failures • temporarily disables failing service to allow recovery rather than overwhelming it with retries.
Graceful Degradation	Return partial results when full task impossible	• Agent completes what it can even when encountering errors • provides best-effort output rather than total failure.
Max-Turns Exceeded	<pre>await Runner.run(agent, input, max_turns=10) # raises MaxTurnsExceeded</pre>	• Raised when an agent loop hits its turn ceiling without a final answer • a run-level failure mode, distinct from a single failed call.
Error Propagation	Pass error context upward in multi-agent hierarchy	• Bubbles failures to supervisor agents who can make recovery decisions • maintains error visibility while delegating handling.

TABLE 15

Evaluation & Testing

Metrics, frameworks, and benchmark suites that production teams use to answer two distinct questions about an agent: *did it succeed?* and *did it succeed for the right reasons?* Outcome-style measures (Task Success Rate, SWE-bench, GAIA) sit alongside trajectory-style measures (Trajectory Analysis, Tool Accuracy) and the human-in-the-loop and LLM-as-Judge graders that score everything in between.

Metric	Example	Description
Task Success Rate	<code>successful_tasks / total_tasks</code>	• Headline outcome metric — fraction of evaluation tasks the agent completes against the success criteria • lets you compare agent versions and benchmark sizes apples-to-apples.
Trajectory Analysis	Evaluate the reasoning path and tool calls, not just the final answer	• Inspects the full transcript: reasoning steps, tool calls, intermediate state • catches agents that pass via lucky paths and reveals <i>why</i> failures happen, not just <i>that</i> they happened.
Tool Accuracy	<code>ToolCorrectnessMetric(threshold=0.5)</code>	• Action-layer metric: did the agent select the right tools with the right arguments? • compares tools actually called against the expected tools; foundational for tool-using agents, since poor tool selection cascades into everything else.
Hallucination Rate	<code>HallucinationMetric(threshold=0.5)</code>	• Frequency of invented information not supported by the provided context • scored with an LLM-as-judge that compares the actual output to the given context; lower is better.
LLM-as-Judge	<code>judge_llm.score(output, rubric)</code>	• Use a stronger LLM to grade outputs against a rubric or pick a winner in a pairwise comparison • scales human evaluation but inherits judge biases like position, verbosity, and self-enhancement.
Human Feedback	Expert review of agent transcripts or user satisfaction ratings	• Gold-standard grading for subjective qualities like helpfulness, tone, and edge-case judgment • expensive and slow, so often reserved for calibrating LLM judges and spot-checking.
SWE-bench	Resolve real GitHub issues from popular Python repos	• Standard benchmark for coding agents: passes only if the patch turns the hidden test suite from failing to passing • Verified is near-saturated in 2026 (top agents ~100%); SWE-bench Pro (multi-language, harder harness, top agent ~46%) is now the harder standard.

Metric	Example	Description
GAIA	466 real-world tasks mixing web browsing, file parsing, multi-document reasoning; top agents ~75% in 2026	• General AI Assistants benchmark chaining tool use, web browsing, and reasoning across three difficulty levels • scores rose from ~20% (2023) to 74.5% (2026); the same model can vary 7 points across orchestration frameworks: the scaffold matters too.
τ-bench	Agent chats with LLM-simulated user AND calls tool APIs; pass ^k measures reliability across k re-runs	• Real-world conversational agent benchmark with domain-specific policies (retail and airline) testing multi-turn interaction, tool use, and rule-following • measures reliability via pass^k: top models drop from ~45-71% on pass¹ to ~25% on pass⁸, revealing production unreliability hidden by single-run averages.
Terminal-Bench	<code>harbor run -d terminal-bench/terminal-bench@latest --agent claude-code --model anthropic/claude-fable-5</code>	• Benchmark for terminal-use agents: realistic sysadmin, build, and reverse-engineering tasks solved from a real shell • a continuous benchmark with tagged releases; frontier agent builders use it to track progress over time.
OSWorld	369 real desktop and web tasks across office apps, browsers, and an IDE, graded by execution-based checks	• Computer-use benchmark grounded in a real Ubuntu desktop rather than a scripted mock • scores multimodal agents with automated, execution-based evaluation functions instead of string matching.
BrowseComp	1,266 fact-finding questions with short, verifiable answers that require browsing many pages	• Benchmark for browsing agents measuring persistence at locating hard-to-find, entangled information • questions are pre-checked unsolvable by GPT-4o (with browsing) and o1 to keep the benchmark hard.
HAL (Holistic Agent Leaderboard)	Princeton's standardized, cost-aware leaderboard covering GAIA, SWE-bench, WebArena, TAU-bench, and more	• Unified cost-aware evaluation harness (accepted ICLR 2026) for reproducible comparison across benchmarks and agent frameworks • tracks the cost-performance Pareto frontier: agents can be 100x more expensive for only 1% accuracy gain, a fact a one-dimensional leaderboard hides.
DeepEval	<code>assert_test(test_case, [ToolCorrectnessMetric()])</code>	• Open-source, pytest-style LLM evaluation framework (Apache 2.0) • ships deterministic metrics (Tool Correctness) alongside LLM-as-Judge metrics (G-Eval, Hallucination, RAGAS, Task Completion).
ScriptedModel	<code>model = ScriptedModel([function_call("get_weather", {"city": "Tokyo"})], [assistant_message("It is sunny in Tokyo.")])</code>	• Swaps the real LLM for a scripted, deterministic stand-in that returns fixed responses in order • tests tool loops, retries, and guardrails without a live model call or API cost.
Benchmark Datasets	AgentBench (OS/DB/web tasks), WebArena (web automation), MMLU (knowledge), HumanEval (code)	• Standardised public test sets that enable apples-to-apples comparison across models • each measures something specific: one benchmark alone never proves general capability; WebArena success rates rose from 15% (2023) to 74.3% (2026); saturated benchmarks lose signal.

TABLE 16

Observability & Debugging

Without observability, multi-step agent failures are nearly impossible to debug: each LLM call, tool invocation, and sub-agent decision happens behind the model's reasoning, and a flat log will not show which step caused the wrong answer. Tracing builds a causal execution tree that dedicated platforms like LangSmith, Langfuse, and Laminar turn into dashboards, replay, and search, and OpenTelemetry's GenAI conventions let that trace data move between them.

Tool	Example	Description
Tracing	<code>@traceable def assistant(q): ...</code>	• Captures the execution tree as parent-child spans across every LLM call, tool use, and sub-agent step • reveals causality and timing that flat logs cannot.

Tool	Example	Description
Logging	<code>logger.info(f"Agent chose tool: {tool_name}")</code>	• Time-stamped records of discrete events written to a persistent store • useful for post-mortem analysis and compliance auditing, but lacks span-level causality.
Real-Time Monitoring	Prebuilt chart: Error rate over time Prebuilt chart: Average latency by model	• Prebuilt dashboards cover trace count, error rate, token usage, and latency for every tracing project automatically • custom dashboards group and filter that data into team-specific views.
LangSmith	<code>export LANGSMITH_TRACING=true</code> <code># traces auto-captured</code>	• LangChain's hosted observability platform with LangGraph Studio IDE for visual agent debugging, 1-click deployment, and zero-setup tracing for LangChain/LangGraph apps • full OTEL support as of March 2026; self-hosting is Enterprise only.
Langfuse	<code>@observe(as_type="agent")</code> <code>def run_agent(q): ...</code>	• Open-source LLM engineering platform (MIT) from an independent team, not LangChain • framework-agnostic via OpenTelemetry with first-class free self-hosting; strong evaluation, prompt management, and dataset workflows.
Laminar	<code>@observe(name="agent")</code> <code>async def run_agent(q):</code> <code>...</code>	• Open-source agent observability platform that automatically catches agent failures and shows them in a readable trace transcript • captures the agent's browser session alongside the trace, with full-text search and raw SQL access across every span.
OpenTelemetry GenAI Conventions	<code>gen_ai.operation.name =</code> <code>invoke_agent</code> <code>gen_ai.agent.name = "Math Tutor"</code>	• Vendor-neutral span and attribute schema for agent operations (create, invoke, plan, execute tool) • adopting it lets one trace move between OTEL-compatible backends instead of locking into one platform's format.
Trace Processors	<code>from agents import</code> <code>add_trace_processor</code> <code>add_trace_processor(MyExporter())</code>	• Built-in SDK instrumentation: every run, agent turn, LLM call, and tool call is auto-wrapped in a trace and nested spans • <code>add_trace_processor()</code> or <code>set_trace_processors()</code> route those spans to your own backend instead of, or alongside, the OpenAI dashboard.
Callback Handlers	<code>on_llm_start</code> , <code>on_llm_end</code> , <code>on_llm_error</code>	• Observer-only event hooks triggered at lifecycle points without modifying the run itself • attach via <code>RunnableConfig</code> for per-request scoping.
Replay	<code>graph.invoke(None,</code> <code>checkpoint_config)</code>	• Re-executes nodes from a saved checkpoint to reproduce a bug or test a fix • in LangGraph, <code>get_state_history</code> lists checkpoints you can replay or fork from.

TABLE 17

Security & Safety

Agentic systems amplify traditional LLM risks because the model can act, hold credentials, and chain tool calls, so a single manipulated prompt or poisoned tool description can escalate into data exfiltration or destructive action. The defenses below come from the OWASP Top 10 for Agentic Applications (2026), OWASP's AI Agent Security Cheat Sheet, and current vendor guardrail toolkits; they work as layered controls, not silver bullets.

Technique	Example	Description
Prompt Injection	Hidden instructions in a retrieved document override the agent's system prompt	• Untrusted content embeds instructions the model treats as commands • direct (in user input) or indirect (in retrieved docs, tool output, images, emails) • OWASP LLM01 and the root vector for most agent breaches.
Input Validation	Strip known injection patterns and segregate untrusted content with delimiters before the model sees it	• Treat every external string as untrusted: user input, retrieved docs, tool output, email bodies • sanitize, length-limit, and clearly mark data vs. instructions • one layer of defense in depth, never sufficient alone.
Memory Poisoning	A poisoned dependency writes itself into an agent's persistent memory and hooks config, then resurfaces as a trusted instruction in every later session	• Attacker content written into memory or shared context keeps steering the agent after the original message is gone • OWASP ASI06, distinct from prompt injection (LLM01), which resets each turn • defend by validating writes and treating retrieved memory as untrusted.

Technique	Example	Description
Tool Poisoning	Malicious MCP server hides instructions inside a tool description the user never sees	• Attacker-controlled tool metadata (descriptions, schemas) manipulates the agent at registration time • a supply-chain-style indirect prompt injection unique to tool registries like MCP • enables "rug pulls" and cross-server shadowing of trusted tools.
Guardrails	<code>nemoguardrails</code> checks input and output flows around every LLM call	• Runtime constraints that wrap the model (input, dialog, retrieval, execution, and output rails) • NeMo Guardrails, Guardrails AI, Llama Guard, Azure Prompt Shields • programmable middleware, not the same as the model's own safety training.
Guardrail Tripwires	<code>if result.tripwire_triggered: raise InputGuardrailTripwireTriggered</code>	• Input guardrails check the user's message before the agent starts; output guardrails check the final answer before it returns • a triggered tripwire raises an exception and halts the run • blocking mode checks first (safer); parallel mode races the agent (faster).
AI Gateway Guardrails	A rule with <code>apply_to: input</code> validates an LLM prompt or an MCP tool's arguments before either reaches the model	• Enforces safety policies once at the gateway layer for all model traffic: PII redaction, prompt injection defense, credential checks • rules apply to LLM prompts and MCP tool calls, on input, output, or both • complements, not replaces, application-level guardrails (Bifrost, Portkey).
Sandboxing	Run untrusted agent code in a gVisor or Firecracker microVM with no host filesystem access	• Isolate agent code execution so a successful exploit cannot reach the host or sensitive data • containers, gVisor user-space kernel, Firecracker/Kata microVMs, WebAssembly • limits blast radius; does not prevent the exploit itself.
Action Approval	Agent proposes <code>transfer_funds</code> ; execution blocks until a human approves the exact parameters	• Human-in-the-loop gate for high-impact or irreversible actions only • bind approval to actor, tool, target, and parameters with a short expiry • applied selectively so routine low-risk actions are not slowed.
Access Control	<code>REVOKE ALL ON orders FROM agent_role;</code> <code>GRANT SELECT (customer_email) ON orders TO agent_role;</code>	• Least-privilege scoping of tool capabilities based on the calling user and session context • per-tool permission lists, read-vs-write splits, scoped tokens • mitigates OWASP LLM06 Excessive Agency.
Non-Human Identity (NHI)	Each agent gets a unique service identity with rotated credentials and an explicit owner	• Treat every agent as its own identity with creation, rotation, and revocation lifecycle • machine identities now outnumber humans by an order of magnitude or more in many enterprises • Entro's 2025 report found 97% of NHIs hold excessive privileges.
Audit Logging	<code>{"tool": "send_email", "params": {"api_key": "***REDACTED***"}, "decision": "approved"}</code>	• Tamper-evident activity trail of decisions, tool calls, approvals, and outcomes • redact secrets before writing, log decision metadata for high-risk actions • powers anomaly detection, incident response, and compliance evidence.
OWASP Agentic Top 10	Goal hijacking, tool misuse, identity abuse, memory poisoning, cascading failures, rogue agents	• Peer-reviewed 2026 risk framework for autonomous agents, published December 2025 • complements the OWASP LLM Top 10 (model-level) with action-level risks • describes a breach progression, not just an isolated checklist.
Agent Governance Toolkit	<code>pip install agent-governance-toolkit</code> ;YAML policy evaluated on every tool call	• Microsoft open-source runtime security layer (April 2026, MIT license) covering all 10 OWASP Agentic risks • stateless policy engine targeting sub-millisecond p99 enforcement • adapters for LangChain, AutoGen, CrewAI, OpenAI Agents SDK, Semantic Kernel.

TABLE 18

Cost Optimization

Token bills scale faster than features once an agent moves to production, so cost control is a first-class engineering concern. An agent loop makes many more LLM calls than a single chatbot turn, and output tokens are billed at a much higher rate than input tokens, so model routing and caching are the highest-leverage levers.

Technique	Example	Description
Model Selection	Route classification to Haiku, save Sonnet/Opus for reasoning	• Task-aware model routing: start efficiency-first with the cheapest model that clears the quality bar, escalating only where evals show a gap • combining a lower-cost model with a frontier model bills most tokens at the lower rate
LLM Cascade	Cheap executor model runs the agent loop; calls a frontier advisor model only on a hard decision, then continues	• Sends each query to a cheap model first, escalating to a stronger model only when the response isn't confident enough • Anthropic's advisor strategy runs this in production: an executor model handles most turns, a frontier advisor handles hard decisions
Prompt Caching	Cache a stable system prompt + tools prefix; vary only the user message	• Reuses repeated prompt portions across calls via <code>cache_control</code> breakpoints • cache reads cost about a tenth of base input, roughly 90% savings; a breakpoint on a block that changes every request only ever writes, never reads
Semantic Caching	Embed query; if cosine similarity to a stored query exceeds the threshold, return the cached answer	• Reuses a prior response when a new query is embedding-similar to one already answered, skipping the model entirely on a hit • distinct from provider-side prompt caching; roughly 31% of LLM queries are semantically similar to an earlier one
Batch API	Submit overnight classification of 10,000 documents	• Asynchronous batch processing at a 50% discount on both input and output tokens • results return within 24 hours (often faster) on OpenAI and Anthropic; suited to evaluations, backfills, and analytics, never real-time user requests
Output Token Limits (max_tokens)	<code>max_tokens=200</code> for summaries vs <code>max_tokens=2000</code> for essays	• Caps generation length; the Chat Completions equivalent is <code>max_completion_tokens</code> • bounds the priciest token class, since output bills at a much higher rate than input; caps generation only, not the input prompt
Early Stopping	<code>AgentExecutor(agent, tools, max_iterations=15, early_stopping_method="force")</code>	• Ends the loop once the agent returns <code>AgentFinish</code>, instead of burning through the iteration budget • on timeout, <code>"force"</code> returns a canned message; <code>"generate"</code> asks the model for one last answer from what it has so far
Iteration Ceiling (max_turns)	<code>Runner.run(agent, input, max_turns=10)</code> # LangGraph: <code>{"recursion_limit": 100}</code>	• Hard cap on agent-loop turns: raises <code>MaxTurnsExceeded</code> (LangGraph: <code>GraphRecursionError</code>) once hit, instead of looping unbounded • a backstop for Early Stopping; <code>max_turns=None</code> (or a high <code>recursion_limit</code>) disables the check
Usage Tracking	<code>usage = result.context_wrapper.usage</code> <code>usage.requests,</code> <code>usage.total_tokens</code>	• Per-run token and request accounting, aggregated automatically across every model call in the run, including calls that produce tool calls or handoffs • the number a budget gate or a FinOps dashboard checks against a limit, instead of hand-rolled counting
LLM FinOps	Tag every call with <code>feature_id + tenant_id</code> ; alert at 50/80/100% of monthly budget	• Applying the financial-operations discipline to AI inference spend • tag-then-aggregate per feature/agent/tenant, then measure cost-per-outcome (resolved ticket, completed task) instead of cost-per-token; without per-agent attribution, kill switches and budgets cannot be enforced

TABLE 19

Production Patterns

Shipping an agent to production exposes failure modes the demo never showed: retried payments creating duplicate charges, an approval gate nobody answers, silent half-completions, and crashed workers losing hours of progress. These are the rails that turn a clever agent into a reliable service: idempotent retries, bounded and batchable human gates, outcome verification, hard time ceilings, and durable state on shutdown.

Pattern	Example	Description
Idempotency	<code>Idempotency-Key: <uuid></code> on a retried POST	• Retry safety for write operations • same key returns the cached first result instead of repeating the side effect • critical for payments, emails, and any tool with external consequences.
Human-in-the-Loop	<code>interrupt()</code> pauses graph before sending an email	• Selective approval gates on high-risk or irreversible actions (5–15% of steps) • agent proceeds autonomously until it hits a gated tool, then waits for <code>Command(resume=...)</code> .
Approval Timeout	<code>this.schedule(86400, "escalateOrReject", approvalId)</code> on a pending gate	• Bounds how long a gate waits for a human response so a pending action never blocks forever • on expiry, escalate to another reviewer or auto-reject, never silently let the action through.
Approval Loop	<code>for r in result.user_input_requests: msg s.append(r.to_function_approval_response(True))</code>	• One review decision covers every gated call raised in that turn instead of one popup per call • the agent collects all pending requests, then resumes with a single batch of responses.
Ground Truth Feedback	After a tool call, read back state and verify the change landed	• Agent gains "ground truth" from the environment after a step (the actual tool result, not just an API status code) • catches "200 OK but row never written" failures and re-plans.
Timeout Management	<code>await asyncio.wait_for(agent(), timeout=60)</code>	• Hard wall-clock ceiling on a single run • cancels the task and raises <code>TimeoutError</code> so runaway loops or hung tool calls can't burn budget forever.
Graceful Shutdown	<code>graceful_shutdown_period=30s</code> lets the in-flight step finish before exit	• Give in-flight work a grace window before the process exits on a shutdown signal • the current task finishes cleanly instead of being killed mid-step, and durable history lets the next worker resume.

References

Official Documentation

1. Augmented LLM
 3. Tool Schema
 5. Tool Result Parsing
 7. Tool Chaining
 9. Reasoning Models
 11. Interleaved Thinking
 13. Short-Term Memory
 15. Structured Note-Taking
 17. Context Window
 19. Tool Result Clearing
 21. Agent Skills (Progressive Disclosure)
 23. Agentic RAG
 25. Sequential
 27. Request-Response
 29. Claude Agent SDK
 31. PydanticAI
 33. Agno
 35. Checkpointing
2. Tool Use
 4. Tool Discovery
 6. Structured Output
 8. Code Execution via MCP
 10. Thinking Budget
 12. To-Do List Tracking
 14. Long-Term Memory
 16. Claude Memory Tool
 18. Compaction
 20. Summarization Middleware
 22. Vector Search
 24. Reranking
 26. AG-UI
 28. CrewAI
 30. Microsoft Agent Framework
 32. smolagents
 34. Semantic Kernel (legacy)
 36. State Persistence

- 37. State Schema
- 39. State Rollback (Fork)
- 41. Asynchronous Execution
- 43. Background Execution
- 45. Retry with Backoff
- 47. Graceful Degradation
- 49. Task Success Rate
- 51. Hallucination Rate
- 53. Real-Time Monitoring
- 55. Langfuse
- 57. Prompt Caching
- 59. Output Token Limits (max_tokens)
- 61. Idempotency
- 63. Approval Loop
- 65. Mem0
- 67. Summarization Middleware
- 69. AG-UI
- 71. OpenAI Agents SDK
- 73. AI Gateway Guardrails
- 75. Access Control
- 77. Iteration Ceiling (max_turns)
- 79. OpenAI Assistants API — Function Calling
- 81. OpenAI Chat Completions — max_completion_tokens
- 83. Anthropic — Claude Models Overview
- 85. LangChain — Agents Concepts
- 87. Microsoft Semantic Kernel — Agents
- 89. mem0 — Documentation
- 91. LlamaIndex — Agentic RAG
- 93. Azure — AI Agent Service Overview

Specifications & Standards

- 94. Prompt Injection
- 95. Input Validation
- 96. Memory Poisoning
- 97. Sandboxing
- 98. OWASP Agentic Top 10
- 99. OWASP — LLM Security Top 10 v2.0

Academic Papers

- 100. Action Space
- 102. Self-Consistency
- 104. ReWOO
- 106. Mem0
- 108. Graph RAG
- 110. LLM-as-Judge
- 112. Zep/Graphiti
- 114. arXiv 2201.11903 — Chain-of-Thought Prompting Elicits Reasoning in LLMs
- 116. arXiv 2303.17760 — HuggingGPT: Solving AI Tasks with ChatGPT
- 118. arXiv 2402.05120 — AgentBench: Evaluating LLMs as Agents
- 120. arXiv 2407.15957 — GAIA: A Benchmark for General AI Assistants
- 122. arXiv 2406.12925 — τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains
- 124. arXiv 2405.14751 — Observational Scaling Laws and the Predictability of Language Model Performance
- 126. arXiv 2308.10848 — MetaGPT: Meta Programming for Multi-Agent Collaboration
- 101. Few-Shot
- 103. Graph-of-Thought
- 105. LLMCompiler
- 107. Letta (MemGPT)
- 109. Blackboard
- 111. LLM Cascade
- 113. arXiv 2210.03629 — ReAct: Synergizing Reasoning and Acting in LLMs
- 115. arXiv 2305.10601 — Tree of Thoughts: Deliberate Problem Solving
- 117. arXiv 2309.02427 — Agents: An Open-source Framework for Autonomous Language Agents
- 119. arXiv 2311.12983 — JARVIS-1: Open-World Multi-Task Agents with Memory-Augmented Multimodal Language Models
- 121. arXiv 2310.11667 — SWE-bench: Can Language Models Resolve Real-World GitHub Issues?
- 123. arXiv 2407.01230 — HAL: A Framework for Holistic Evaluation of AI Agents
- 125. arXiv 2309.07864 — Cognitive Architectures for Language Agents (CoALA)
- 127. arXiv 2304.03442 — Generative Agents: Interactive Simulacra of Human Behavior

- 128. arXiv 2401.05268 — WebArena: A Realistic Web Environment for Building Autonomous Agents
- 130. arXiv 2406.06608 — SWE-bench Verified
- 132. arXiv 2410.02107 — OpenAI o1 System Card

- 134. arXiv 2503.01214 — Computer-Using Agents: Survey and Benchmark Analysis

Books

- 135. Chip Huyen — AI Engineering (O'Reilly, 2025)

Source Code & Repositories

- 136. Zep/Graphiti
- 138. Swarm
- 140. Terminal-Bench
- 142. OpenTelemetry GenAI Conventions
- 144. AI Gateway Guardrails
- 146. Sandboxing
- 148. Anthropic Cookbook — Tool Use Examples
- 150. CrewAI — GitHub
- 152. OpenAI Agents SDK — GitHub
- 154. NVIDIA NeMo Guardrails — GitHub
- 156. Mem0 — GitHub
- 158. browser-use — GitHub
- 160. AgentBench — GitHub
- 162. HAL Leaderboard — GitHub
- 164. Pydantic AI — GitHub
- 166. Semantic Kernel — GitHub
- 168. OpenTelemetry GenAI SIG — GitHub
- 170. Microsoft Agent Governance Toolkit — GitHub
- 172. Phidata — GitHub
- 174. Temporal AI Workflows — GitHub
- 176. E2B — GitHub

Articles & Tutorials

- 177. Function Calling
- 179. Perception
- 181. Computer-Using Agent (CUA)
- 183. Hosted Tools vs Function Tools
- 185. Zero-Shot
- 187. Agentic Reasoning
- 189. Hierarchical Task Network (HTN)
- 191. Reflection
- 193. Working Memory
- 195. Procedural Memory
- 197. Prompt Compression
- 199. Agent Handoff
- 201. Loop (Iterative)
- 203. Agent-to-Agent (A2A)
- 205. LangGraph
- 207. OpenAI Agents SDK
- 209. goose
- 211. Sessions
- 213. Circuit Breaker
- 215. GAIA
- 217. OSWorld
- 219. HAL (Holistic Agent Leaderboard)
- 221. Benchmark Datasets
- 223. Laminar
- 225. Callback Handlers

- 129. arXiv 2301.04589 — Toolformer: Language Models Can Teach Themselves to Use Tools
- 131. arXiv 2404.01349 — LLM Agent Operating System (AIOS)
- 133. arXiv 2412.04093 — From Solo Performance to Symphony: AI Agents in Multi-Agent Systems

- 137. Hierarchical
- 139. Strands Agents
- 141. DeepEval
- 143. Guardrails
- 145. AutoGen (legacy)
- 147. Microsoft UFO — Windows Desktop Agent
- 149. LangGraph — GitHub
- 151. OpenAI Swarm (experimental) — GitHub
- 153. Hugging Face smolagents — GitHub
- 155. Guardrails AI — GitHub
- 157. MemoryOS — GitHub
- 159. WebArena — GitHub
- 161. SWE-bench — GitHub
- 163. Instructor — GitHub
- 165. DSPy — GitHub
- 167. Toolhouse — GitHub
- 169. MCP Inspector — GitHub
- 171. CAMEL — GitHub
- 173. Agno (formerly Phidata) — GitHub
- 175. Composio — GitHub

- 178. Agentic Workflow
- 180. Reasoning Engine
- 182. Autonomy
- 184. Chain-of-Thought
- 186. Tree-of-Thought
- 188. Task Decomposition
- 190. ReAct
- 192. Plan-and-Execute
- 194. Episodic Memory
- 196. AGENTS.md
- 198. Query Transformation
- 200. Collaborative
- 202. Model Context Protocol (MCP)
- 204. Pub-Sub
- 206. LangChain
- 208. Google ADK
- 210. AutoGen (legacy)
- 212. Sync Execution
- 214. SWE-bench
- 216. τ -bench
- 218. BrowseComp
- 220. ScriptedModel
- 222. Logging
- 224. Trace Processors
- 226. Tool Poisoning

- 227. [Guardrail Tripwires](#)
- 229. [Agent Governance Toolkit](#)
- 231. [Usage Tracking](#)
- 233. [Zep/Graphiti](#)
- 235. [Model Context Protocol \(MCP\)](#)
- 237. [Model Context Protocol \(MCP\)](#)
- 239. [AG-UI](#)
- 241. [goose](#)
- 243. [LangGraph — Checkpointing](#)
- 245. [AutoGen — Multi-Agent Conversation Framework](#)
- 247. [Google DeepMind — AlphaCode 2 Technical Report](#)
- 249. [AWS Strands Agents — Documentation](#)
- 251. [Weaviate — What Are AI Agents?](#)
- 253. [Tungsten Automation — Agentic AI in Production 2026](#)
- 255. [Zylos — AI Agent Cost Optimization and Token Economics 2026](#)
- 257. [Mem0 — State of AI Memory 2026](#)
- 259. [Evidently AI — AI Agent Evaluation Benchmarks 2026](#)
- 261. [Laminar — AI Observability with Browser Session Replay](#)
- 263. [Maxim AI — Best AI Guardrails Platforms in 2026](#)
- 265. [VentureBeat — AI Agents Fail 1 in 3 Production Deployments](#)
- 267. [MorphLLM — Framework Comparison 2026](#)
- 269. [Langfuse — Tracing and Observability for LLM Apps](#)
- 271. [Arize Phoenix — LLM Tracing](#)
- 273. [Simon Willison — LLM Tool Use and Function Calling](#)
- 275. [Lilian Weng — LLM-Powered Autonomous Agents](#)
- 277. [Weights & Biases — Agent Evaluation with Weave](#)
- 279. [Context Engineering Guide 2026](#)
- 281. [Towards Data Science — ReAct Agent Pattern Explained](#)
- 283. [PromptingGuide.AI — ReAct Prompting](#)
- 285. [E2B — Code Interpreter for AI Agents](#)
- 287. [Stanford HAI — AI Index Report 2026](#)
- 289. [AWS — Agentic AI Reference Architecture](#)
- 291. [Hamel Husain — Your AI Product Needs Evals](#)
- 293. [Gartner — Agentic AI Hype Cycle 2025](#)
- 295. [a16z — The Current State of Agentic AI](#)
- 297. [InfoQ — AI Agent Architecture Patterns Guide 2026](#)
- 228. [Non-Human Identity \(NHI\)](#)
- 230. [Semantic Caching](#)
- 232. [LLM FinOps](#)
- 234. [Letta \(MemGPT\)](#)
- 236. [Model Context Protocol \(MCP\)](#)
- 238. [Agent-to-Agent \(A2A\)](#)
- 240. [Google ADK](#)
- 242. [Anthropic — Model Context Protocol Specification](#)
- 244. [LangGraph — Multi-Agent Architectures](#)
- 246. [Microsoft Azure — Computer-Using Agents \(CUA\)](#)
- 248. [AWS — Durable AI Agents with LangGraph and DynamoDB](#)
- 250. [AWS Strands Agents — Swarm Multi-Agent Pattern](#)
- 252. [Vellum — AI Agent Architectures 2026](#)
- 254. [Zylos — CUA State of Computer-Using Agents 2026](#)
- 256. [Vectorize.io — AI Memory Systems Comparison 2026](#)
- 258. [Sierra — \$\tau\$ -bench: A Benchmark for Tool-Agent-User Interaction](#)
- 260. [Latitude — LangSmith vs LangFuse vs Helicone: Observability Comparison 2026](#)
- 262. [WorkOS — MCP 2026: Current State and Future](#)
- 264. [Kili Technology — Guide to AI Agent Benchmarks 2026](#)
- 266. [Developers Digest — Multi-Agent Coordination Patterns 2026](#)
- 268. [Evidently AI — LLM Evaluation Guide](#)
- 270. [Helicone — LLM Monitoring](#)
- 272. [Portkey AI — AI Gateway](#)
- 274. [DeepLearning.AI — Multi-Agent Systems Course](#)
- 276. [OpenAI Cookbook — Tool Use Examples](#)
- 278. [Braintrust — LLM Evaluation Platform](#)
- 280. [Towards AI — Agentic RAG Patterns](#)
- 282. [Towards Data Science — Chain of Thought Prompting](#)
- 284. [AssemblyAI — LLM Agents Tutorial](#)
- 286. [GAIA Benchmark — Hugging Face](#)
- 288. [Google — Agents Companion: Foundational Guide](#)
- 290. [Eugene Yan — Patterns for Building LLM-based Systems and Products](#)
- 292. [CISA — Guidelines for Secure AI System Development](#)
- 294. [McKinsey — The State of AI 2026](#)
- 296. [Sequoia Capital — AI Agents Market Map 2026](#)

Video

- 298. [Andrej Karpathy — Intro to Large Language Models \(YouTube\)](#)
- 300. [DeepLearning.AI — Functions, Tools, and Agents with LangChain \(YouTube\)](#)
- 302. [Matt Wolfe — AI Agents in 2026 Overview \(YouTube\)](#)
- 304. [LangChain — LangGraph Tutorial \(YouTube\)](#)
- 299. [Andrew Ng — Agentic AI Design Patterns \(YouTube\)](#)
- 301. [AI Explained — OpenAI o1 and Chain of Thought \(YouTube\)](#)
- 303. [Yannic Kilcher — ReAct Paper Explained \(YouTube\)](#)
- 305. [Microsoft Research — AutoGen Multi-Agent Systems \(YouTube\)](#)